

Chapter 18: Probabilistic Surrogate Models

Algorithms for Optimization, 2nd Edition (2026)
Kochenderfer & Wheeler

Lecture Slides

June 12, 2026

Chapter Overview

- 1 Motivation
- 2 18.1 Gaussian Distribution
- 3 18.2 Gaussian Processes
- 4 18.3 Prediction (Posterior Mean and Variance)
- 5 18.4 Gradient Measurements
- 6 18.5 Noisy Measurements
- 7 18.6 Fitting Gaussian Processes (MLE)
- 8 Summary & Exercises

Why Probabilistic Surrogate Models?

Key Idea

When optimizing expensive black-box functions, we want not just a *prediction* but also a measure of *confidence*.

- Deterministic surrogates (polynomial, RBF) give predictions but no uncertainty quantification.
- **Gaussian processes (GPs)** represent a probability distribution over functions.
- A GP posterior gives a **mean prediction** and a **variance** at every query point.
- Uncertainty can drive intelligent exploration (Bayesian optimization, Chapter 19).

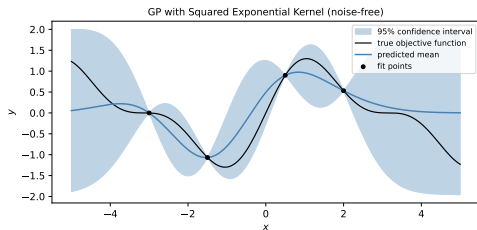


Figure: GP posterior mean and 95% confidence interval fitted to four noise-free evaluations.

18.1 Multivariate Gaussian Distribution

Definition (Eq. 18.1)

An n -dimensional **Gaussian distribution** is parametrised by mean $\mu \in \mathbb{R}^n$ and positive semi-definite covariance matrix $\Sigma \in \mathbb{R}^{n \times n}$:

$$\mathcal{N}(\mathbf{x} \mid \mu, \Sigma) = (2\pi)^{-n/2} |\Sigma|^{-1/2} \exp\left(-\frac{1}{2}(\mathbf{x}-\mu)^\top \Sigma^{-1}(\mathbf{x}-\mu)\right)$$

We write $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$.

Covariance matrix structure:

- Diagonal Σ_{ii} : variance of each component
- Off-diagonal Σ_{ij} : covariance between components i, j
- Σ must be positive semi-definite

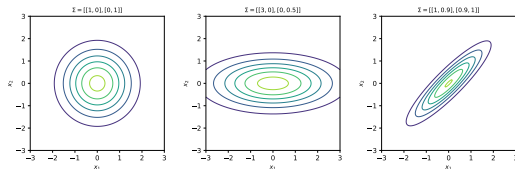


Figure: Contour plots of 2D Gaussians with different Σ : identity (circular), diagonal (ellipse), and correlated.

Joint, Marginal, and Conditional Gaussians

Joint distribution (Eq. 18.3)

Two jointly Gaussian random variables **a** and **b**:

$$\begin{bmatrix} \mathbf{a} \\ \mathbf{b} \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \boldsymbol{\mu}_a \\ \boldsymbol{\mu}_b \end{bmatrix}, \begin{bmatrix} \mathbf{A} & \mathbf{C} \\ \mathbf{C}^\top & \mathbf{B} \end{bmatrix}\right)$$

Marginal distributions (Eq. 18.4)

$$\mathbf{a} \sim \mathcal{N}(\boldsymbol{\mu}_a, \mathbf{A}), \quad \mathbf{b} \sim \mathcal{N}(\boldsymbol{\mu}_b, \mathbf{B})$$

Sum out the other variable.

Conditional distribution (Eqs. 18.5–18.7)

$$\mathbf{a} \mid \mathbf{b} \sim \mathcal{N}(\boldsymbol{\mu}_{a|b}, \boldsymbol{\Sigma}_{a|b})$$

$$\boldsymbol{\mu}_{a|b} = \boldsymbol{\mu}_a + \mathbf{CB}^{-1}(\mathbf{b} - \boldsymbol{\mu}_b)$$

$$\boldsymbol{\Sigma}_{a|b} = \mathbf{A} - \mathbf{CB}^{-1}\mathbf{C}^\top$$

Key fact

Both the marginal and conditional distributions of a multivariate Gaussian are themselves Gaussian — an extremely convenient closed-form property.

Conditional Gaussian – Numerical Example (Ex. 18.1)

Setup: Joint distribution over (x_1, x_2) :

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 1 & 1.5 \end{bmatrix}\right)$$

Marginals:

$$x_1 \sim \mathcal{N}(0, 2), \quad x_2 \sim \mathcal{N}(0, 1.5)$$

Conditional $x_1 \mid x_2 = 1$:

$$\mu_{1|2} = 0 + \frac{1}{1.5}(1 - 0) = \frac{2}{3}$$

$$\sigma_{1|2}^2 = 2 - \frac{1^2}{1.5} = \frac{4}{3}$$

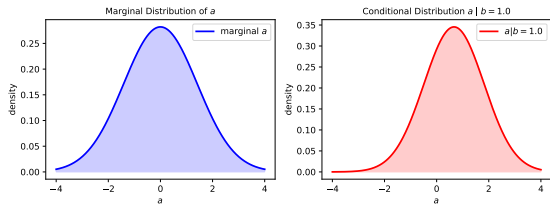


Figure: Marginal distribution of x_1 (left) vs. conditional distribution $x_1 \mid x_2 = 1$ (right). Conditioning shifts the mean and reduces variance.

18.2 Gaussian Processes

Definition

A **Gaussian process** $\mathcal{GP}(m, k)$ is a probability distribution over *functions* $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

- **Mean function** $m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$ (often zero)
- **Covariance/kernel function** $k(\mathbf{x}, \mathbf{x}') = \text{Cov}(f(\mathbf{x}), f(\mathbf{x}'))$

Finite-dimensional view: For any finite set of points $X = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$, the values $[f(\mathbf{x}^{(1)}), \dots, f(\mathbf{x}^{(m)})]^\top$ follow a multivariate Gaussian:

$$\mathbf{y} \sim \mathcal{N}(\mathbf{m}(X), \mathbf{K}(X, X))$$

where $[\mathbf{K}(X, X)]_{ij} = k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$.

Properties:

- Nonparametric: complexity grows with data
- Fully determined by m and k
- Analytically tractable posterior
- Computational cost $O(m^3)$ (matrix inversion)

Kernel (Covariance) Functions

The kernel $k(\mathbf{x}, \mathbf{x}')$ encodes our prior belief about function smoothness.

Kernel	Formula
Constant	c_f^2
Linear	$\sum_d c_d^2 x_d x'_d$
Polynomial	$(x^\top x' + c_p^2)^p$
Exponential	$\exp(-r)$
γ -Exponential	$\exp(-(r/\ell)^\gamma)$
Squared Exp.	$\exp\left(-\frac{r^2}{2\ell^2}\right)$
Matérn	$\frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}}{\ell} r\right)^\nu K_\nu\left(\frac{\sqrt{2\nu}}{\ell} r\right)$
Rat. Quadratic	$\left(1 + \frac{r^2}{2\alpha\ell^2}\right)^{-\alpha}$

$$r = \|\mathbf{x} - \mathbf{x}'\|$$

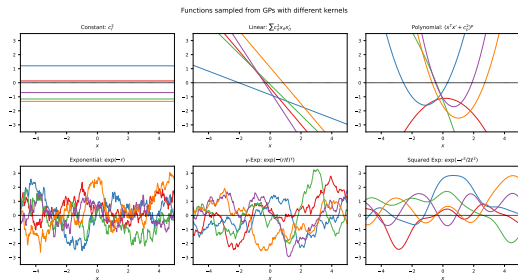


Figure: Functions sampled from GPs with different kernels. Smoother kernels produce smoother sample paths.

Kernel Functions in Python

```
import numpy as np

def sq_exp_kernel(x1, x2, ell=1.0, sigma_f=1.0):
    """Squared exponential (RBF) kernel."""
    r2 = np.sum((x1 - x2)**2)
    return sigma_f**2 * np.exp(-r2 / (2 * ell**2))

def matern52_kernel(x1, x2, ell=1.0):
    """Matern 5/2 kernel."""
    r = np.sqrt(np.sum((x1 - x2)**2))
    t = np.sqrt(5) * r / ell
    return (1 + t + t**2 / 3) * np.exp(-t)

def rat_quad_kernel(x1, x2, ell=1.0, alpha=1.0):
    """Rational quadratic kernel."""
    r2 = np.sum((x1 - x2)**2)
    return (1 + r2 / (2 * alpha * ell**2))**(-alpha)

# Build kernel matrix K(X, X)
def build_K(X, kernel, noise=0.0):
    m = len(X)
    K = np.array([[kernel(X[i], X[j]) for j in range(m)]
                  for i in range(m)])
    return K + (noise + 1e-8) * np.eye(m)
```

Key Property: Squared Exponential Kernel

Squared Exponential (SE) kernel

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right)$$

- ℓ : **length-scale** (horizontal scale)
- Corresponds to smooth (C^∞) functions
- **Stationarity**: depends only on $\mathbf{x} - \mathbf{x}'$
- $k(\mathbf{x}, \mathbf{x}) = 1$ (unit variance prior)

Interpretation: Two points close together ($\|\mathbf{x} - \mathbf{x}'\| \ll \ell$) are highly correlated; far-apart points ($\|\mathbf{x} - \mathbf{x}'\| \gg \ell$) are nearly independent.

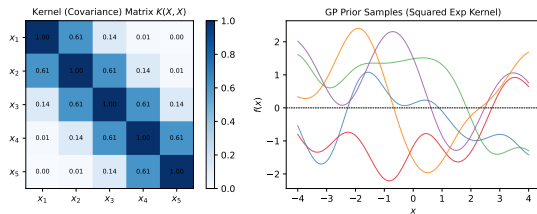


Figure: Kernel (covariance) matrix for 5 equally-spaced points (left) and GP prior samples (right).

18.3 GP Prediction – Setup

Given

- Training data: m observations $(X, \mathbf{y})$ where $X = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ and $\mathbf{y} = [y^{(1)}, \dots, y^{(m)}]^\top$
- Query (test) points: X^* (ℓ points)
- GP prior: $\mathcal{GP}(m(\cdot), k(\cdot, \cdot))$

Joint prior distribution (Eq. 18.10):

$$\begin{bmatrix} \hat{\mathbf{y}} \\ \mathbf{y} \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mathbf{m}(X^*) \\ \mathbf{m}(X) \end{bmatrix}, \begin{bmatrix} \mathbf{K}(X^*, X^*) & \mathbf{K}(X^*, X) \\ \mathbf{K}(X, X^*) & \mathbf{K}(X, X) \end{bmatrix} \right)$$

Apply the conditional Gaussian formula (Eqs. 18.5–18.7) to obtain the **posterior** $\hat{\mathbf{y}} \mid \mathbf{y}$.

18.3 GP Posterior (Noise-Free)

Posterior distribution (Eqs. 18.13–18.16)

$$\hat{\mathbf{y}} \mid \mathbf{y} \sim \mathcal{N}(\hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\Sigma}})$$

$$\hat{\boldsymbol{\mu}} = \mathbf{m}(X^*) + \mathbf{K}(X^*, X)^\top \mathbf{K}(X, X)^{-1} (\mathbf{y} - \mathbf{m}(X))$$

$$\hat{\boldsymbol{\Sigma}} = \mathbf{K}(X^*, X^*) - \mathbf{K}(X^*, X)^\top \mathbf{K}(X, X)^{-1} \mathbf{K}(X^*, X)$$

Symbol guide:

- $\mathbf{K}(X^*, X)$: $\ell \times m$ cross-covariance matrix
- $\mathbf{K}(X, X)$: $m \times m$ training covariance matrix
- $\mathbf{K}(X^*, X^*)$: $\ell \times \ell$ test covariance matrix
- Posterior is also referred to as the *posterior distribution*

Predictive variance at a single point $\mathbf{x}^*$:

$$\hat{\sigma}(\mathbf{x}^*) = k(\mathbf{x}^*, \mathbf{x}^*) - \mathbf{k}^\top \mathbf{K}(X, X)^{-1} \mathbf{k}$$

Complexity:

- Matrix inversion $\mathbf{K}(X, X)^{-1}$: $O(m^3)$
- Prediction at ℓ points: $O(m^2 \ell)$
- Bottleneck: m^3 grows fast — sparse/approximate GP methods address this at scale

95% confidence region (Eq. 18.17):

$$\hat{\mu}(\mathbf{x}) \pm 1.96 \hat{\sigma}(\mathbf{x})$$

GP Prediction in Python (Algorithm 18.2)

```
import numpy as np

def gp_predict(X_train, y_train, X_pred, kernel, noise=0.0):
    """
    Gaussian process prediction.
    Returns posterior mean mu and variance var at X_pred.
    """
    m = len(X_train)
    n = len(X_pred)

    # Build covariance matrices
    K_XX = np.array([[kernel(X_train[i], X_train[j])
                      for j in range(m)] for i in range(m)])
    K_XX += (noise + 1e-8) * np.eye(m)           # regularise

    K_Xs = np.array([[kernel(X_pred[i], X_train[j])
                      for j in range(m)] for i in range(n)])

    K_ss = np.array([[kernel(X_pred[i], X_pred[j])
                      for j in range(n)] for i in range(n)])

    # Posterior
    K_inv = np.linalg.solve(K_XX, np.eye(m))    #  $O(m^3)$ 
    mu     = K_Xs @ K_inv @ y_train             # posterior mean
    cov    = K_ss - K_Xs @ K_inv @ K_Xs.T       # posterior covariance
    var    = np.diag(cov)                       # marginal variances
```

GP Prediction – Numerical Example

Setup:

- Kernel: SE with $\ell = 1$, $\sigma_f = 1$
- Training points: $x \in \{-3, -1.5, 0.5, 2.0\}$
- True function: $f(x) = \sin(x) + 0.5 \sin(2x)$
- Zero mean GP prior

Observations:

- Posterior mean interpolates training data exactly
- Uncertainty (shaded region) is small near training points
- Uncertainty grows far from training data
- Expected value $\rightarrow 0$ as $x \rightarrow \pm\infty$

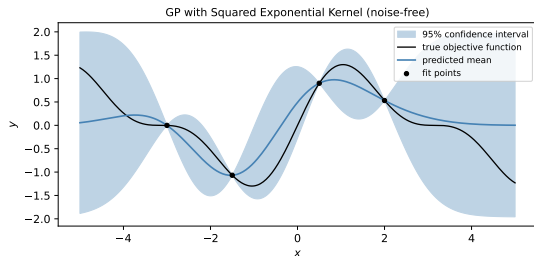


Figure: GP posterior with 95% confidence interval.

2D Gaussian Process

GPs naturally extend to multi-dimensional design spaces.

- Same formulas apply; $\mathbf{x} \in \mathbb{R}^n$
- Kernel $k(\mathbf{x}, \mathbf{x}')$ must be positive semi-definite
- SE kernel: $k(\mathbf{x}, \mathbf{x}') = \exp(-\frac{1}{2\ell^2} \|\mathbf{x} - \mathbf{x}'\|^2)$
- Cost scales as $O(m^3)$ regardless of n

Note: Covariance matrix size is $m \times m$ (number of observations), *not* related to input dimension n .

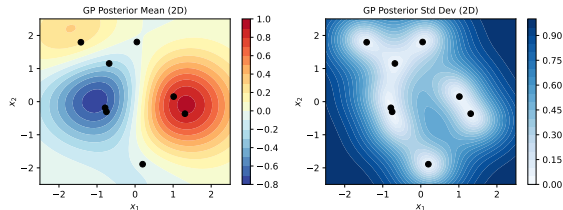


Figure: GP posterior mean (left) and standard deviation (right) fitted to 8 random observations in 2D. Black dots mark training points.

18.4 Gradient Measurements – Motivation

Key insight

Gradient information is *free* in many applications (automatic differentiation, adjoint methods). Each gradient observation adds n constraints, significantly reducing uncertainty.

Extended GP model (Eq. 18.18):

$$\begin{bmatrix} \mathbf{y} \\ \nabla \mathbf{y} \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mathbf{m}_f \\ \mathbf{m}_{\nabla} \end{bmatrix}, \begin{bmatrix} \mathbf{K}_{ff} & \mathbf{K}_{f\nabla} \\ \mathbf{K}_{\nabla f} & \mathbf{K}_{\nabla\nabla} \end{bmatrix} \right)$$

where each covariance block is derived from the base kernel k .

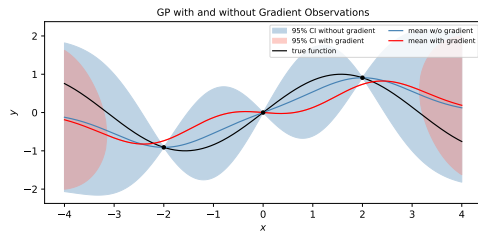


Figure: GP with (salmon) and without (blue) gradient observations. Gradient information significantly tightens the confidence interval.

Covariance Functions with Gradients (Eqs. 18.19–18.22)

Derived covariance functions (via linearity of covariance)

$$k_{ff}(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}, \mathbf{x}') \quad (\text{original kernel})$$

$$k_{\nabla f}(\mathbf{x}, \mathbf{x}')_i = \nabla_{\mathbf{x}_i} k(\mathbf{x}, \mathbf{x}')$$

$$k_{f\nabla}(\mathbf{x}, \mathbf{x}')_i = \nabla_{\mathbf{x}'_i} k(\mathbf{x}, \mathbf{x}')$$

$$k_{\nabla\nabla}(\mathbf{x}, \mathbf{x}')_{ij} = \nabla_{\mathbf{x}_i} \nabla_{\mathbf{x}'_j} k(\mathbf{x}, \mathbf{x}')$$

Example 18.2: SE kernel derivatives

For $k_{ff}(\mathbf{x}, \mathbf{x}') = \exp(-\frac{1}{2}\|\mathbf{x} - \mathbf{x}'\|^2)$:

$$k_{\nabla f}(\mathbf{x}, \mathbf{x}')_i = -(x_i - x'_i) \exp(-\frac{1}{2}\|\mathbf{x} - \mathbf{x}'\|^2)$$

$$k_{\nabla\nabla}(\mathbf{x}, \mathbf{x}')_{ij} = -((i = j) - (x_i - x'_i)(x_j - x'_j)) \exp(-\frac{1}{2}\|\mathbf{x} - \mathbf{x}'\|^2)$$

where $(i = j)$ evaluates to 1 if true, 0 if false.

GP Posterior with Gradients (Eqs. 18.25–18.27)

Joint distribution for prediction (Eq. 18.23):

$$\begin{bmatrix} \hat{\mathbf{y}} \\ \mathbf{y} \\ \nabla \mathbf{y} \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mathbf{m}_f(X^*) \\ \mathbf{m}_f(X) \\ \mathbf{m}_{\nabla}(X) \end{bmatrix}, \begin{bmatrix} \mathbf{K}_{ff}(X^*, X^*) & \mathbf{K}_{ff}(X^*, X) & \mathbf{K}_{f\nabla}(X^*, X) \\ \mathbf{K}_{ff}(X, X^*) & \mathbf{K}_{ff}(X, X) & \mathbf{K}_{f\nabla}(X, X) \\ \mathbf{K}_{\nabla f}(X, X^*) & \mathbf{K}_{\nabla f}(X, X) & \mathbf{K}_{\nabla\nabla}(X, X) \end{bmatrix} \right)$$

Posterior (same formula as before, Eqs. 18.25–18.27)

$$\begin{aligned} \hat{\mathbf{y}} \mid \mathbf{y}, \nabla \mathbf{y} &\sim \mathcal{N}(\boldsymbol{\mu}_{\nabla}, \boldsymbol{\Sigma}_{\nabla}) \\ \boldsymbol{\mu}_{\nabla} &= \mathbf{m}_f(X^*) + \begin{bmatrix} \mathbf{K}_{ff}(X^*, X) \\ \mathbf{K}_{\nabla f}(X^*, X) \end{bmatrix}^{\top} \begin{bmatrix} \mathbf{K}_{ff}(X, X) & \mathbf{K}_{f\nabla}(X, X) \\ \mathbf{K}_{\nabla f}(X, X) & \mathbf{K}_{\nabla\nabla}(X, X) \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{y} - \mathbf{m}_f(X) \\ \nabla \mathbf{y} - \mathbf{m}_{\nabla}(X) \end{bmatrix} \end{aligned}$$

Covariance block dimensions (Eq. 18.24): $\ell \times \ell$, $\ell \times m$, $\ell \times nm$, $m \times m$, $m \times nm$, $nm \times nm$ where ℓ = query pts, m = obs, n = input dim.

Gradient GP in Python

```
import numpy as np

def kff(x, xp, ell=1.0):
    return np.exp(-0.5 * (x - xp)**2 / ell**2)

def kvf(x, xp, ell=1.0):    #  $d/dx k(x, xp)$ 
    return -(x - xp) / ell**2 * kff(x, xp, ell)

def kvv(x, xp, ell=1.0):    #  $d^2/dx^2 k(x, xp)$ 
    d = x - xp
    return (1.0/ell**2 - d**2/ell**4) * kff(x, xp, ell)

def gp_predict_with_gradients(X_obs, y_obs, dy_obs, X_pred, ell=1.0):
    m, n = len(X_obs), len(X_pred)
    # Build extended training covariance [K_ff K_fv; K_vf K_vv]
    K_ff = np.array([[kff(X_obs[i], X_obs[j], ell) for j in range(m)] for i in range(m)])
    K_vf = np.array([[kvf(X_obs[i], X_obs[j], ell) for j in range(m)] for i in range(m)])
    K_vv = np.array([[kvv(X_obs[i], X_obs[j], ell) for j in range(m)] for i in range(m)])
    K_tr = np.block([[K_ff, K_vf.T], [K_vf, K_vv]]) + 1e-8*np.eye(2*m)
    # Cross-covariances to prediction points
    Ksf = np.array([[kff(X_pred[i], X_obs[j], ell) for j in range(m)] for i in range(n)])
    Ksv = np.array([[kvf(X_pred[i], X_obs[j], ell) for j in range(m)] for i in range(n)])
    Ks = np.hstack([Ksf, Ksv])
    # Posterior mean
    y_comb = np.concatenate([y_obs, dy_obs])
    mu = Ks @ np.linalg.solve(K_tr, y_comb)
    return mu
```

18.5 Noisy Measurements

Model (Eq. 18.28)

Observations include zero-mean Gaussian noise:

$$y = f(\mathbf{x}) + z, \quad z \sim \mathcal{N}(0, \nu)$$

$$\begin{bmatrix} \hat{\mathbf{y}} \\ \mathbf{y} \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mathbf{m}(X^*) \\ \mathbf{m}(X) \end{bmatrix}, \begin{bmatrix} \mathbf{K}(X^*, X^*) & \mathbf{K}(X^*, X) \\ \mathbf{K}(X, X^*) & \mathbf{K}(X, X) + \nu \mathbf{I} \end{bmatrix} \right)$$

Posterior (Eqs. 18.29–18.31):

$$\hat{\mathbf{y}} \mid \mathbf{y}, \nu \sim \mathcal{N}(\boldsymbol{\mu}^*, \boldsymbol{\Sigma}^*)$$

$$\boldsymbol{\mu}^* = \mathbf{m}(X^*) + \mathbf{K}(X^*, X)(\mathbf{K}(X, X) + \nu \mathbf{I})^{-1}(\mathbf{y} - \mathbf{m}(X))$$

$$\boldsymbol{\Sigma}^* = \mathbf{K}(X^*, X^*) - \mathbf{K}(X^*, X)(\mathbf{K}(X, X) + \nu \mathbf{I})^{-1}\mathbf{K}(X, X^*)$$

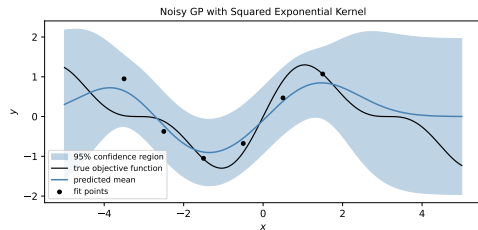


Figure: Noisy GP with SE kernel. The posterior does *not* exactly interpolate data; the noise ν controls the balance between fit and smoothness.

Noisy GP Prediction in Python (Algorithm 18.4)

```
import numpy as np

def sq_exp_kernel(x1, x2, ell=1.0):
    return np.exp(-0.5 * (x1 - x2)**2 / ell**2)

def predict_noisy_gp(X_train, y_train, X_pred,
                    kernel, noise_var=0.1):
    """
    GP prediction with Gaussian observation noise.
    noise_var: variance nu of noise  $z \sim N(0, \nu)$ 
    Returns: mu_pred, var_pred
    """
    m = len(X_train)
    n = len(X_pred)

    K_XX = np.array([[kernel(X_train[i], X_train[j])
                      for j in range(m)] for i in range(m)])
    K_XX += (noise_var + 1e-8) * np.eye(m)  # add noise

    K_Xs = np.array([[kernel(X_pred[i], X_train[j])
                      for j in range(m)] for i in range(n)])
    K_ss_diag = np.array([kernel(X_pred[i], X_pred[i])
                          for i in range(n)])

    alpha = np.linalg.solve(K_XX, y_train)
    mu = K_Xs @ alpha
    tmp = np.linalg.solve(K_XX, K_Xs.T)
```

Noise-Free vs. Noisy GP: Key Differences

Noise-free GP:

- $\mathbf{K}(X, X) + \epsilon \mathbf{I}$ (small ϵ for numerical stability)
- Posterior **exactly interpolates** training data
- Posterior variance is zero at training points
- Appropriate when function evaluations are deterministic

Noisy GP:

- $\mathbf{K}(X, X) + \nu \mathbf{I}$ (larger ν = more noise)
- Posterior **does not interpolate** — smoothes through data
- Non-zero variance even at training points
- Appropriate for experimental data, simulation with randomness
- ν can be tuned via MLE (Section 18.6)

Practical tip

Always add a small nugget term ($\epsilon \approx 10^{-8}$) for numerical stability when inverting the kernel matrix, even in the noise-free case.

18.6 Fitting GP Hyperparameters via MLE

Log marginal likelihood (Eq. 18.34)

GP parameters θ (length-scale ℓ , signal variance σ_f^2 , noise ν) are fitted by maximizing:

$$\log p(\mathbf{y} \mid X, \theta) = -\frac{1}{2} \mathbf{y}^\top \mathbf{K}^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K}| - \frac{m}{2} \log 2\pi$$

where $\mathbf{K} = \mathbf{K}(X, X; \theta) + \nu \mathbf{I}$.

Three terms:

- 1 **Data fit:** $-\frac{1}{2} \mathbf{y}^\top \mathbf{K}^{-1} \mathbf{y}$ — rewards models that explain the data
- 2 **Complexity penalty:** $-\frac{1}{2} \log |\mathbf{K}|$ — penalises overly flexible models (Occam's razor)
- 3 **Normalisation:** constant

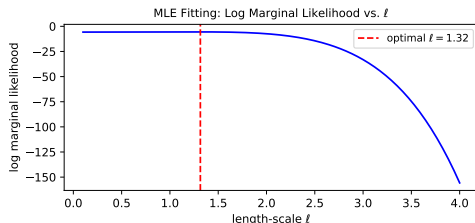


Figure: Log marginal likelihood as a function of the length-scale ℓ . The optimal ℓ balances fit and smoothness.

Gradient of the Log Marginal Likelihood

Gradient w.r.t. θ_j (Eq. 18.35)

$$\frac{\partial}{\partial \theta_j} \log p(\mathbf{y} \mid X, \boldsymbol{\theta}) = \frac{1}{2} \mathbf{y}^\top \mathbf{K}^{-1} \frac{\partial \mathbf{K}}{\partial \theta_j} \mathbf{K}^{-1} \mathbf{y} - \frac{1}{2} \text{tr} \left(\mathbf{K}^{-1} \frac{\partial \mathbf{K}}{\partial \theta_j} \right)$$

Numerical considerations:

- Use Cholesky decomposition for stability:
 $\mathbf{K} = \mathbf{L}\mathbf{L}^\top$
- $\log |\mathbf{K}| = 2 \sum_i \log L_{ii}$
- $\mathbf{K}^{-1} \mathbf{y}$ via triangular solves
- Gradient enables gradient-based optimisation (L-BFGS-B)
- Multiple random restarts to avoid local optima

Typical hyperparameters:

Parameter	Role
ℓ	length-scale (smoothness)
σ_f^2	signal variance (amplitude)
ν	noise variance

Optimise in log-space to enforce positivity:
 $\theta_j = \exp(\tilde{\theta}_j)$.

MLE Fitting in Python

```
import numpy as np
from scipy.optimize import minimize

def log_marginal_likelihood(params, X_train, y_train):
    log_ell, log_sf, log_nu = params
    ell = np.exp(log_ell); sf = np.exp(log_sf); nu = np.exp(log_nu)

    m = len(X_train)
    K = sf**2 * np.array([
        [np.exp(-0.5*((X_train[i]-X_train[j])/ell)**2)
          for j in range(m)] for i in range(m)])
    K += (nu + 1e-8) * np.eye(m)

    try:
        L = np.linalg.cholesky(K)
    except np.linalg.LinAlgError:
        return np.inf                # non-PD: reject

    alpha = np.linalg.solve(L.T, np.linalg.solve(L, y_train))
    log_det = 2.0 * np.sum(np.log(np.diag(L)))
    lml = -0.5 * (y_train @ alpha + log_det
                  + m * np.log(2*np.pi))

    return -lml                      # negate for minimisation

# Optimise with multiple restarts
best = None
for _ in range(5):
```

Summary

Key concepts:

- 1 **Multivariate Gaussian** has closed-form marginal and conditional distributions — the foundation of GPs.
- 2 **Gaussian Process** = distribution over functions, characterised by mean $m(\mathbf{x})$ and kernel $k(\mathbf{x}, \mathbf{x}')$.
- 3 **Kernel selection** determines smoothness; SE is the most common choice.
- 4 **GP prediction**: posterior mean $\hat{\mu}$ and covariance $\hat{\Sigma}$ via conditional Gaussian formula. Cost: $O(m^3)$.
- 5 **Gradient measurements**: extend the GP using derivative kernel functions; significantly reduces uncertainty.
- 6 **Noisy observations**: add $\nu \mathbf{I}$ to training covariance; posterior no longer interpolates.
- 7 **MLE fitting**: maximise log marginal likelihood

Complexity summary:

Operation	Cost
Build $\mathbf{K}(X, X)$	$O(m^2)$
Invert $\mathbf{K}(X, X)$	$O(m^3)$
Predict at ℓ points	$O(m^2 \ell)$
With gradients (nm obs.)	$O((nm)^3)$
MLE gradient	$O(m^3)$ per step

Key advantage

GPs are **nonparametric**: their complexity adapts to the data, unlike fixed-basis regression models.

Exercise 18.3 – Derivative Information Comparison

Problem: $f(x) = \frac{\sin(x)}{x^2 + 1}$ on $[-5, 5]$.

Fit points: $\{-5, -2.5, 0, 2.5, 5\}$.

Compare GP with and without derivative info.

Derivative:

$$f'(x) = \frac{(x^2 + 1) \cos(x) - 2x \sin(x)}{(x^2 + 1)^2}$$

Covariance functions (SE kernel, $\ell = 1$):

$$k_{ff}(x, x') = \exp\left(-\frac{1}{2}\|x - x'\|^2\right)$$

$$k_{\nabla f}(x, x') = (x' - x) \exp\left(-\frac{1}{2}\|x - x'\|^2\right)$$

$$k_{\nabla \nabla}(x, x') = ((x - x')^2 - 1) \exp\left(-\frac{1}{2}\|x - x'\|^2\right)$$

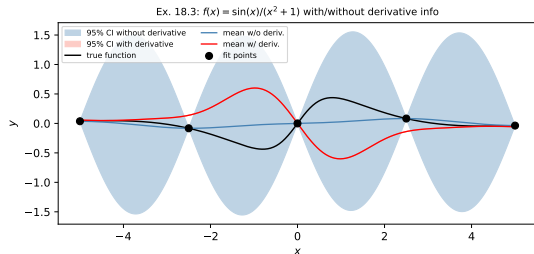


Figure: With derivative info (salmon), confidence interval is tighter. Max. std with derivative ≈ 0.377 at $x \approx \pm 3.8$. At least 8 points needed without derivatives to match this accuracy.

Exercise 18.10 – Kernel Selection via Cross-Validation

Problem: Select best kernel for data $\{(1, 0), (2, -1), (3, -2), (4, 1), (5, 0)\}$ using leave-one-out (LOO) cross-validation.

Maximise the sum of LOO log-likelihoods across all folds.

Results:

Kernel	Log-lik.
$\exp(-\ x - x'\)$	-8.688
$\exp(-\ x - x'\ ^2)$	-9.010
$(1 + \ x - x'\)^{-1}$	-9.579
$(1 + \ x - x'\ ^2)^{-1}$	-10.195
$(1 + \ x - x'\)^{-2}$	-8.088

Best: Rational quadratic $(1 + \|x - x'\|)^{-2}$.

LOO CV procedure:

- 1 For each fold i ($i = 1 \dots 5$):
 - Hold out point (x_i, y_i)
 - Fit GP on remaining 4 points
 - Evaluate $\log p(y_i \mid x_i, \text{other data})$
- 2 Sum log-likelihoods over folds
- 3 Select kernel with highest total

Key insight: LOO-CV is a principled model selection criterion that avoids overfitting to the training set.

Key Takeaways I

Gaussian Distribution (Sec. 18.1)

- n -dimensional Gaussian parametrised by μ and Σ
- Marginal: retain a subset of variables – still Gaussian
- Conditional: condition on some variables – still Gaussian
- Closed-form formulas for both are the engine of GP inference

Gaussian Processes (Sec. 18.2)

- Distribution over functions, defined by mean function $m(\mathbf{x})$ and kernel function $k(\mathbf{x}, \mathbf{x}')$
- Kernel choice controls smoothness of function samples
- Nonparametric: model complexity grows with data

Key Takeaways II

Prediction (Sec. 18.3)

- Posterior mean and variance at query points computed analytically
- Uncertainty grows far from training data, approaches prior far away
- Prediction cost $O(m^3)$; dominant cost is matrix inversion

Key Takeaways III

Gradient Measurements (Sec. 18.4)

- Derivative observations seamlessly incorporated via derived kernels
- Gradient covariance functions derived by differentiating the kernel
- Each gradient observation adds n constraints — powerful when gradients are cheap

Noisy Measurements (Sec. 18.5)

- Model noise as $y = f(\mathbf{x}) + z$, $z \sim \mathcal{N}(0, \nu)$
- Equivalent to adding $\nu \mathbf{I}$ to training kernel matrix
- Posterior no longer interpolates — smooths through noisy data

Key Takeaways IV

Fitting GPs (Sec. 18.6)

- Maximise log marginal likelihood to learn kernel hyperparameters
- Balances data fit vs. model complexity (Bayesian Occam's razor)
- Use gradient-based optimisation (L-BFGS-B) with multiple restarts
- Can also use leave-one-out cross-validation for kernel selection

Further Reading

Textbook:

- Rasmussen & Williams, *Gaussian Processes for Machine Learning*, MIT Press, 2006 (freely available online)
- Kochenderfer & Wheeler, *Algorithms for Optimization*, 2nd ed., MIT Press, 2026

Libraries (Python):

- scikit-learn: GaussianProcessRegressor
- GPy: full-featured GP library
- GPyTorch: GPU-accelerated GPs (PyTorch)
- BoTorch: Bayesian optimisation with GPs

Next chapter:

Chapter 19 – Surrogate Optimisation
(Bayesian optimisation using GP posteriors)

- Acquisition functions (EI, UCB, PI)
- Exploration vs. exploitation trade-off
- Optimising the acquisition function